Repositories application setup and persistence

Arnav Sankapal | Roll 23206 | SY10 / E10

Your explanation in one minute

I own the repository interfaces, application entry point and shared project setup. Spring Data JPA generates the repository implementation. My queries load users, lock balance rows and filter transaction history. The H2 file database stores data across restarts.

Files and concepts you own

File	What to understand
UserRepository.java	Extends JpaRepository<WalletUser, Long>. It finds users by email, checks duplicates, reads an ID without loading the balance, and locks a user row for updates.
TransactionRepository.java	Uses a JPQL query to return the initiating user's attempts and a recipient's successful incoming transfers, ordered by time and then ID descending.
DigitalWalletApplication.java	Contains main and @SpringBootApplication. SpringApplication.run starts the application context and embedded server.
Build and application configuration	pom.xml supplies dependencies and Java 17 targeting. The wrapper downloads Maven. application.properties selects file-backed H2 and local server/session settings.

How your code connects

Aadi supplies the entities. Chandurwar and Patidar call UserRepository. Patidar calls the history query and locking query inside service transactions. Charu supplies test properties separately so tests use an in-memory database.

The whole application in one flow

Request -> Spring Security -> Controller -> Service -> Repository -> H2. The response returns JSON or a Thymeleaf HTML view. Your module is one part of this application; this ZIP is not a standalone runnable project.

All original file paths are listed in FILES_TO_PUSH.txt. Copy only the contents of project-files into the repository root. The frontend archive contains no Java. Source line references refer to this package snapshot.

Read the code in this order

1 Generated query methods

findByEmail and existsByEmail follow Spring Data naming rules. Optional<WalletUser> makes a missing result explicit. You write the interface; Spring supplies a bean at runtime.

2 JPQL and parameters

The @Query strings refer to Java entity names and properties, not raw SQL table/column names. @Param binds a method argument to a named placeholder such as :email or :userId.

3 Read the ID before locking

findIdByEmail selects only u.id. Loading the full entity first could place an old balance in JPA's persistence context. WalletService asks for the ID, then loads the entity through the locked query.

4 Database row lock

@Lock(PESSIMISTIC_WRITE) on findByIdForUpdate requests a database lock. It must be used inside a transaction. Locks last until the transaction completes. WalletService locks the smaller user ID first to reduce deadlock risk.

5 History and storage

The history query includes every attempt initiated by the user, but incoming records only when SUCCESS. The H2 URL uses a file in data/, so changing the working directory changes the database location. Keep data/ out of Git; never delete it to fix a code issue.

Actual source excerpt: repository/UserRepository.java, lines 13-19

```
13      @Query("select u.id from WalletUser u where u.email = :email")
14      Optional<Long> findIdByEmail(@Param("email") String email);
15
16      // A database row lock makes another balance update wait for this transaction.
17      @Lock(LockModeType.PESSIMISTIC_WRITE)
18      @Query("select u from WalletUser u where u.id = :id")
19      Optional<WalletUser> findByIdForUpdate(@Param("id") Long id);
```

Trace a real operation

WalletService needs to transfer from A to B. It requests both IDs using email, sorts the IDs, and asks findByIdForUpdate for each row. While those locks are held, another writer to the same wallet waits or encounters a timeout. The service checks funds and commits both balances and the record. History queries then expose the permitted records to each user.

Viva questions and answers

Practise explaining the reasoning in your own words. Open the named class while you answer.

1. Why is the repository an interface with no implementation?

Spring Data JPA creates a proxy implementation based on JpaRepository, method names and @Query declarations.

2. What do WalletUser and Long mean in JpaRepository?

WalletUser is the entity type. Long is the Java type of its primary key.

3. What is the difference between JPQL and SQL?

JPQL queries entities and their Java properties. SQL queries physical database tables and columns.

4. Why does the history query check SUCCESS for the recipient?

Failed outgoing attempts belong to the initiator. Recipients should see completed incoming transfers, not every failed attempt addressed to them.

5. Does @Transactional alone prevent overspending?

Not necessarily. Two transactions could read the same old balance. The write lock makes the read-check-update sequence wait for competing writers.

6. Can a lock wait forever?

This H2 configuration sets LOCK_TIMEOUT=10000 milliseconds. A conflict can surface as a locking exception; the application advises checking history before retrying.

7. Why order by both createdAt and id?

Newest timestamps come first. The ID is a second ordering key for records with equal timestamps.

8. What does ddl-auto=update do?

Hibernate attempts to update the schema from entity mappings at startup. It is convenient for this mini project, not a replacement for managed production migrations.

9. Why does the app find classes below com.pict.wallet?

The application class sits in the root package. @SpringBootApplication enables auto-configuration and component scanning beneath that package, with Boot discovering the entities and repositories there.

10. Why did the earlier packaged demo return HTTP 500?

The running JVM was using a JAR that a build replaced. It then failed to load classes. Stop a packaged process before rebuilding it, or run a separate immutable JAR copy.

Upload and practise

Use your own GitHub identity

Ask the coordinator to invite you as a collaborator. Clone the shared repository after main has its initial README. Replace REPOSITORY_URL and YOUR_VERIFIED_EMAIL below. Git identity is configured only for this repository.

```
git clone REPOSITORY_URL pocket-wallet
cd pocket-wallet
git config user.name "Arnav Sankapal"
git config user.email "YOUR_VERIFIED_EMAIL"
git switch -c backend/repositories-sankapal
```

Now copy everything inside this ZIP's project-files folder into the root of that clone, merging the src folders. Do not copy the outer member folder or flatten the Java package folders. Stage only your listed files:

```
git add -- src/main/java/com/pict/wallet/repository/UserRepository.java
git add -- src/main/java/com/pict/wallet/repository/TransactionRepository.java
git add -- src/main/java/com/pict/wallet/DigitalWalletApplication.java
# Stage the remaining paths in FILES_TO_PUSH.txt as well.

git diff --cached
git commit -m "Add repositories application setup and database configuration"
git push -u origin backend/repositories-sankapal
```

Open a pull request into main. The coordinator should preserve each member's authored commits when merging. Initial partial uploads can have missing-class errors; run the full build after all seven packages are assembled. If identical code already exists, copying it produces no new change.

Checks to practise

1. Explain all four custom user lookup methods and which ones load an entity versus only an ID.

2. Create an account, add simulated funds, stop and restart the full application, and confirm the same balance and history remain.

3. Locate simultaneousTransfersCannotOverspend and oppositeDirectionTransfersUseConsistentLockOrder in WalletServiceTest.

4. Upload the shared build files early so everyone has the same project structure. A partial set of modules will not compile yet.

After assembly: ./mvnw verify | Focused tests: ./mvnw -Dtest=WalletServiceTest test

GitHub links for attribution details: docs.github.com/en/account-and-profile/how-tos/email-preferences/setting-your-commit-email-address and the troubleshooting missing contributions article linked in START_HERE.md. A commit email must be linked to the member's account; merely using another person's display name is not enough.